

SQLite-Carver-Pro

Manuel de Référence Technique & Guide d'Utilisation

Version 1.2.0 — Forensic Analysis Toolkit

⚠ Avertissement & Obligation de Vérification Forensique :

Le carving binaire et la récupération de données dans le slack space reposent sur des heuristiques d'analyse de motifs pour reconstruire des cellules non référencées ou partiellement écrasées. **Tout artefact ou enregistrement reconstruit doit impérativement être vérifié, corroboré et recoupé** avec les journaux systèmes ou les sources primaires avant toute conclusion d'investigation ou d'expertise légale. Le logiciel est fourni sans garantie.

Domaines couverts :

- Algorithmique du Carving SQLite de bas niveau & Optimisations O(1)
- Explication mathématique du Seuil de Confiance (0.0 à 1.0)
- Freeblocks, Slack Space, Espace non alloué & Freelist
- Récupération résiliente des chaînes de texte tronquées dans le Slack
- Filtrage strict des faux positifs Protobuf & Décodage zlib/bplist
- Timeline WAL interactive & Diffs visuels avant/après (avec filtre --table)
- Support Multilingue (Français / Anglais) & Routeur d'export unifié

Auteur : Dam-FOR3K (avec l'assistance technique d'Antigravity / Google DeepMind)

Compatibilité : Windows 64-bit / Linux / macOS (Python 3.11+ & Binaire autonome .exe)

Date du document : Août 2026 (Édition v1.2.0)

Table des Matières

1. Avertissement Légal & Obligation de Vérification

2. Introduction & Architecture Globale

3. Fonctionnement Interne de SQLite & Mécanismes de Carving

- 3.1. Anatomie physique d'un fichier SQLite (.db)
- 3.2. Types de pages B-Tree & Encodage des Varints
- 3.3. Les 4 zones de carving forensique (Calcul exact des intervalles de Slack)
- 3.4. Reconstruction des chaînes de débordement (Overflow)
- 3.5. Journal Write-Ahead Log (WAL) & Fusion automatique dans Carve
- 3.6. Décodage automatique des payloads binaires imbriqués (Apple bplist, Protobuf, zlib)

4. Comprendre le Seuil de Confiance Forensique (Confidence Threshold)

- 4.1. Pourquoi un seuil de confiance ?
- 4.2. Formule mathématique, Invariants SQLite & Barème des structures imbriquées
- 4.3. Guide d'interprétation des 4 paliers de confiance
- 4.4. Comment choisir le bon seuil selon le scénario d'enquête

5. Guide d'Utilisation de la Ligne de Commande & du .exe

- 5.1. Inspection de la base (`info`)
- 5.2. Carving des données supprimées & Détection WAL (`carve`)
- 5.3. Recherche forensique récursive (`search`)
- 5.4. Différentiel des transactions (`wal-diff` avec filtre table)
- 5.5. Microscope binaire indépendant avec métadonnées (`decode-blob`)

6. Rapports Forensiques & Formats d'Export

- 6.1. Le Tableau de Bord HTML Interactif à Double Vue (All Evidence & Timeline WAL avec Pagination 60 FPS et Switcher Bilingue)
- 6.2. Format JSON Indenté (`.json`) & JSON Lines (`.jsonl`)
- 6.3. Format Tabulaire Excel (`.csv`) & Parquet (`.parquet`)

7. API Python pour Pipelines d'Investigation

1. Avertissement Légal & Obligation de Vérification

⚠ Recommandation Méthodologique Fondamentale

Dans toute analyse numérique, **aucune décision critique, opérationnelle ou judiciaire ne doit être prise sur la seule base d'une reconstruction heuristique automatisée**. Les enregistrements récupérés dans les freeblocks, le slack space ou les journaux WAL non validés peuvent avoir été partiellement altérés par des écritures ultérieures. L'analyste doit impérativement corroborer les données extraites par d'autres sources de preuves indépendantes.

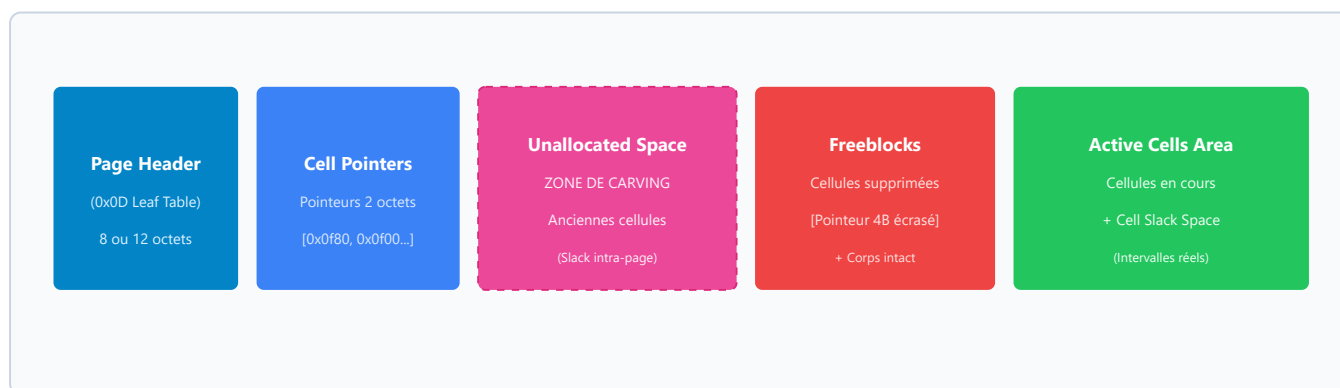
2. Introduction & Architecture Globale

Dans les analyses forensiques, les bases de données SQLite constituent l'un des conteneurs de preuves les plus fréquents (historiques de navigation, messageries chiffrées de type Signal/Telegram/WhatsApp, applications mobiles iOS/Android, journaux d'événements système, tokens d'authentification).

Les outils de carving traditionnels se limitent généralement à une extraction de chaînes de caractères brutes (Greedy Text Dump), ignorant la structure relationnelle, les schémas de tables, les types de colonnes, et les structures binaires complexes sérialisées au sein des colonnes BLOB.

SQLite-Carver-Pro v1.2.0 a été conçu comme un moteur forensique complet de niveau système capable de disséquer la structure B-Tree binaire, de retrouver les résidus de cellules dans les zones non allouées et les freeblocks, de recoller les chaînes de débordement multi-pages et de déchiffrer les structures imbriquées (Apple Binary Plist, Google Protocol Buffers, flux compressés zlib/deflate).

Anatomie Visuelle d'une Page SQLite B-Tree Leaf (4096B)



3. Fonctionnement Interne de SQLite & Mécanismes de Carving

3.1. Anatomie physique d'un fichier SQLite (.db)

Un fichier SQLite est découpé en pages de taille fixe (généralement 4096 octets, ou 512 à 65536 octets). La page 1 contient l'en-tête de base de données de 100 octets.

3.2. Types de pages B-Tree & Encodage des Varints

Les enregistrements SQLite utilisent des **Varints** (entiers à longueur variable de 1 à 9 octets en Big-Endian) et une table de **Serial Types** pour déterminer le type et la taille de chaque colonne :

Serial Type	Type de Donnée & Longueur	Signification Forensique
0	0 octet	Valeur <code>NULL</code> .
1, 2, 3, 4, 5, 6	1, 2, 3, 4, 6, 8 octets	Entiers signés (avec extension de signe pour entiers 24-bit et 48-bit).
7	8 octets	Nombre à virgule flottante IEEE 754 (64-bit float).
8, 9	0 octet	Constante numérique entière 0 ou 1 (optimisation SQLite).
$N \geq 12$ (Pair)	$(N - 12) / 2$ octets	Données binaires brutes BLOB (ex: images, bplist, protobuf).
$N \geq 13$ (Impair)	$(N - 13) / 2$ octets	Chaîne de caractères TEXT encodée en UTF-8.

4. Comprendre le Seuil de Confiance Forensique (Confidence Threshold)

4.1. Pourquoi un seuil de confiance ?

Lorsqu'une base de données SQLite est corrompue ou lorsqu'on explore l'espace non alloué et les freeblocks, le disque dur contient des milliers d'octets résiduels. Si l'on applique un décodeur sans validation stricte, n'importe quelle séquence aléatoire d'octets pourrait être interprétée à tort comme un enregistrement SQLite valide (faux positif).

Pour garantir une **fiabilité d'analyse**, `sqlite-carver-pro` attribue à chaque enregistrement récupéré un **Score de Confiance compris entre 0.00 et 1.00**.

4.2. Formule Mathématique de Calcul du Score & Barème des Structures Imbriquées

Le score de confiance est calculé dynamiquement selon 5 piliers stricts :

1. Invariant de Longueur de Payload (Physique) :

La formule physique $\text{taille_payload} = \text{taille_header} + \text{somme}(\text{longueurs})$ doit être rigoureusement respectée. Lors de la découverte d'une cellule valide, le pointeur de scan avance exactement de la taille totale de la cellule pour éviter tout chevauchement ou doublon fantôme.

2. Validation des Caractères Imprimables (Sanity Check) :

Pour chaque colonne identifiée comme du `TEXT` (serial type impair ≥ 13), le moteur vérifie que 100% des octets correspondent à des caractères UTF-8 / ASCII valides et imprimables. Si du bruit binaire non décodable est présent, le score chute drastiquement.

3. Appariement Heuristique de Schéma & Score Intrinsèque :

Le moteur compare la séquence de types trouvée avec les schémas découverts dans `sqlite_schema`. Si aucun schéma n'est disponible (ex: table orpheline ou détruite), un score intrinsèque dynamique est calculé en fonction de la cohérence des types et de la lisibilité des textes (annoté explicitement comme *unmapped schema* dans les détails).

4. Pénalité d'En-tête Écrasé (Incomplete Record Penalty) :

Dans les *freeblocks*, SQLite écrase les 4 premiers octets avec son pointeur de liste chaînée. L'outil reconstruit la cellule à partir de l'offset + 4 mais applique une pénalité structurelle (score initial fixé à `0.85`).

5. Barème Calibré des Structures Imbriquées (BLOBs) :

Pour éviter l'inflation artificielle de confiance sur des formats sans en-tête magique :

- **Apple Binary Plist (`bp1list00`)** : `1.00` (Signature magique stricte + parsing certifié).
- **Flux Zlib / Deflate + Texte clair** : `0.95` (Validation CRC/Adler32 + UTF-8 imprimable).
- **Flux Zlib + Protobuf ou Binaire** : `0.85` (Décompression CRC validée).
- **Google Protocol Buffers (Multi-champs)** : `0.85` (Validation des wire types et des numéros de champs).
- **Google Protocol Buffers (Champ unique / Isolé)** : `0.65` (Attribution prudente pour éliminer les faux positifs).

4.3. Guide d'Interprétation des 4 Paliers de Confiance

< 0.50 (Bruit / Rejeté)

0.50 - 0.64 (Slack)

0.65 - 0.84 (Freeblock)

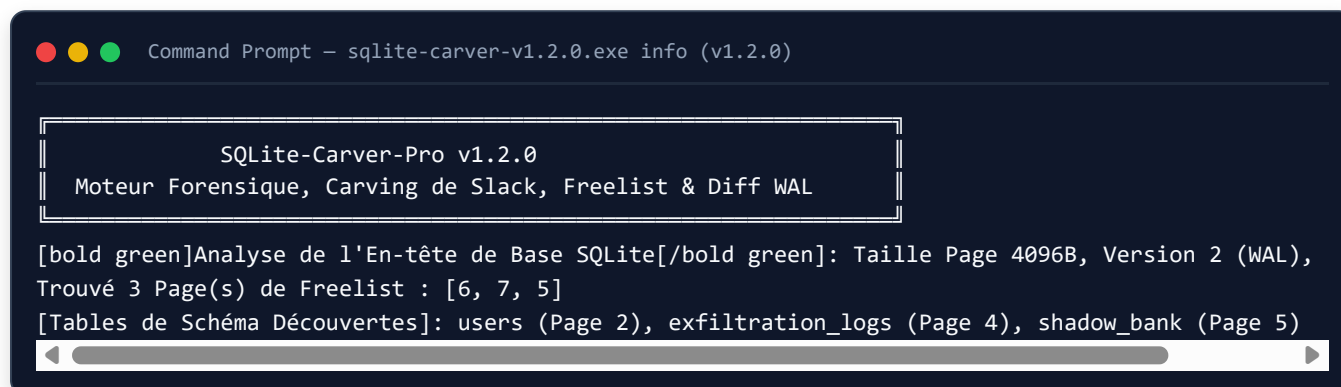
0.85 - 1.00 (Haute)

Plage de Score	Niveau de Confiance	Origine Typique & Signification pour l'Enquêteur
0.85 – 1.00	Haute Fidélité (Certitude Maximale)	Cellules actives ou cellules récemment supprimées intactes. Correspondance parfaite avec le schéma de table, types stricts respectés, textes 100% lisibles.
0.65 – 0.84	Bonne Confiance (Lignes Reconstituées)	Cellules issues des Freeblocks dont le pointeur de début a été écrasé, ou lignes avec colonnes NULL/partielles, ou structures Protobuf sans signature.
0.50 – 0.64	Confiance Modérée (Slack & Reliques)	Fragments de cellules trouvés dans le Slack Space ou les Freelist Pages orphelines (tables ayant subi un <code>DROP TABLE</code> dont le schéma a disparu de <code>sqlite_schema</code>).
< 0.50	Bruit Binaire (Rejeté par défaut)	Artefacts fragmentés, octets aléatoires de clés de chiffrement ou résidus de compression. Filtrés automatiquement pour éviter d'inonder le rapport.

5. Guide d'Utilisation de la Ligne de Commande & du .exe

5.1. Inspection de la base (info)

```
# Diagnostic bas niveau, détection des freelists et tables (avec choix de langue)
sqlite-carver-v1.2.0.exe info evidence.db --lang fr
```



5.2. Carving des données supprimées & Détection WAL (carve)

Détection Automatique du Journal WAL : Le journal WAL compagnon est pris en compte automatiquement par défaut. Si le fichier WAL est corrompu ou illisible, un avertissement explicite est affiché dans la console.

```
# 1. Carving complet (actifs + supprimés + journal WAL compagnon)
sqlite-carver-v1.2.0.exe carve evidence.db

# 2. Isoler UNIQUEMENT les enregistrements supprimés (Freeblocks, Freelist, Slack, WAL)
sqlite-carver-v1.2.0.exe carve evidence.db --deleted-only

# 3. Filtrer par seuil de confiance strict (ex: >= 0.80 pour exclure le slack trop fragmenté)
sqlite-carver-v1.2.0.exe carve evidence.db --min-confidence 0.80

# 4. Exporter le tableau de bord web interactif complet avec onglet Timeline WAL
sqlite-carver-v1.2.0.exe carve evidence.db --export rapport_investigation.html
```

5.3. Recherche forensique récursive (search)

```
# Recherche récursive par défaut incluant base + journal WAL
sqlite-carver-v1.2.0.exe search evidence.db "session_token_xyz"

# Désactiver explicitement l'inclusion du journal WAL
sqlite-carver-v1.2.0.exe search evidence.db "token" --no-include-wal

# Recherche d'un hash ou pattern hexadécimal dans les champs BLOB
sqlite-carver-v1.2.0.exe search evidence.db "deadbeef" --hex
```

5.4. Différentiel des transactions (wal-diff)

```
# Reconstitution complète de l'historique chronologique des transactions
sqlite-carver-v1.2.0.exe wal-diff evidence.db evidence.db-wal

# Cibler une table suspecte spécifique
sqlite-carver-v1.2.0.exe wal-diff evidence.db evidence.db-wal --table users --export diff_users.html
```

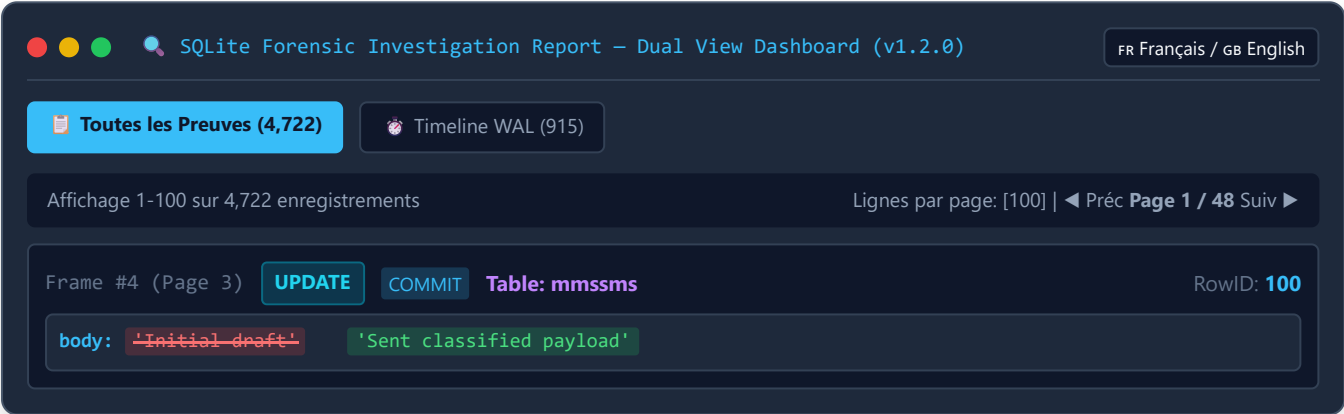
6. Rapports Forensiques & Formats d'Export

6.1. Le Tableau de Bord HTML Interactif à Double Vue (All Evidence & Timeline WAL avec Pagination 60 FPS et Sélecteur Bilingue)

L'export HTML génère un **Tableau de Bord Forensique Interactif 100% Autonome (Air-Gapped)** qui s'ouvre directement dans n'importe quel navigateur (Edge, Chrome, Firefox) sans aucune installation requise.

Moteur de Rendu Ultra-Fluide (60 FPS) : Grâce à une pré-indexation en mémoire et à une pagination dynamique (50, 100, 250, 500 lignes ou Tout), le rapport reste instantané même sur des bases volumineuses de plusieurs dizaines de milliers d'enregistrements.

```
sqlite-carver-v1.2.0.exe carve evidence.db --export rapport_investigation.html
```



6.2. Guide des Formats d'Export

Format	Extension	Usage & Public Cible
Rapport HTML	.html	Recommandé pour l'enquêteur. Dashboard interactif bilingue avec recherche temps réel, curseur de confiance, pagination 60 FPS, onglet Timeline WAL et dépliage des BLOBs.
JSON Indenté	.json	Structure claire et aérée avec retours à la ligne pour analyse manuelle sous VS Code / Notepad++.
JSON Lines	.jsonl	Format streaming (une ligne par enregistrement) pour ingestion SIEM (Splunk, ElasticSearch, Cortex XSOAR).
Tableur CSV	.csv	Analyse tabulaire sous Microsoft Excel, LibreOffice Calc, ou importation dans des bases SQL externes (inclut la colonne <code>details</code>).
Apache Parquet	.parquet	Export binaire compressé haute performance pour data science forensique (requêtes Polars, DuckDB, Pandas).

7. API Python pour Pipelines d'Investigation

```
from pathlib import Path
from sqlite_carver import SQLiteCarver, WalDiffEngine, ForensicSearchEngine, export_html

# 1. Carving programmatique d'une base brute (v1.2.0)
raw_db = Path("evidence.db").read_bytes()
carver = SQLiteCarver(raw_db)

# 2. Extraction avec filtre de confiance (ex: 0.80+)
records = [r for r in carver.carve_all() if r.confidence >= 0.80]
for r in records:
    if r.source != "active":
        print(f"[{r.source.upper()}] Page {r.page_id} | Table: {r.matched_table} | Conf: {r.confidence}")

# 3. Recherche récursive de preuves
engine = ForensicSearchEngine(raw_db)
matches = engine.search("session_token_xyz")
print(f"Trouvé {len(matches)} occurrences du token !")

# 4. Génération automatique du rapport HTML interactif
export_html(records, "rapport_automatise.html")
```

Bonnes Pratiques d'Analyse Recommandées

1. Conservez toujours l'intégrité de la preuve en travaillant exclusivement sur une copie de sauvegarde en lecture seule.
2. Placez toujours les fichiers compagnons `.db-wal` et `.db-shm` dans le même dossier pour l'analyse temporelle.
3. Utilisez le format `.html` pour vos rapports : il permet aux juristes et analystes d'explorer les preuves en toute autonomie.